

TOUR MANAGER MVP - ESPECIFICAÇÃO TÉCNICA COMPLETA

Sistema de Gestão Financeira, Comercial, RH e Operacional

Versão: 1.0

Data: 25/12/2025

Cliente: Emerson - HARDCASE

Projeto: Expansão do sistema TOUR Manager

ÍNDICE

1. Visão Geral
 2. Contexto do Projeto
 3. Arquitetura Atual
 4. Novos Módulos
 5. Módulo Financeiro Expandido
 6. Módulo Comercial com IA
 7. Módulo RH
 8. Módulo Manutenção
 9. Inteligência Artificial
 10. Automações
 11. Interface e UX
 12. Plano de Implementação
 13. Anexos
-

1. VISÃO GERAL

Objetivo

Expandir o sistema TOUR Manager MVP existente com módulos completos de gestão financeira, comercial (CRM), recursos humanos e inteligência artificial para recomendações estratégicas.

Escopo

Sistema web Flask multi-tenant (SQLite) focado em mobile-first para gestão de: - **Receitas:** Faturas, NFSe, PIX, contratos - **Despesas:** Contas fixas/variáveis, folha CLT, freelancers, manutenção - **Provisionamento:** Sistema automático de previsão financeira - **CRM:** Funil de vendas, leads, orçamentos, reativação - **RH:** Funcionários CLT, freelancers, folha de pagamento - **IA:** Assistente financeiro e comercial com recomendações estratégicas - **Dashboards:** Gráficos analíticos, comparativos temporais, alertas

Princípios de Desenvolvimento

1. **NÃO QUEBRAR CÓDIGO EXISTENTE** - Toda nova funcionalidade é aditiva
2. **MÓDULOS INDEPENDENTES** - Cada blueprint funciona isoladamente
3. **MIGRATIONS SEGURAS** - Sempre usar ALTER TABLE IF NOT EXISTS
4. **CÓDIGO BASE** - Análise completa antes de modificações
5. **MOBILE FIRST** - Interface otimizada para smartphones/tablets

2. CONTEXTO DO PROJETO

Sistema Atual (Funcionando)

- Flask + SQLAlchemy + Jinja2
- Multi-tenant (company_id em todas tabelas)
- Módulos: Equipment, Tour, Separation List, Scanner (QR/NFC/RFID)
- Financial básico: Quote, Invoice, Payment
- Integração: Asaas (PIX/Boleto), Focus NFe

Estrutura de Dados Existente

```
Company (multi-tenant)
├── User (roles: admin/user)
├── Equipment (QR/NFC/RFID)
│   └── Category → Type → Brand → Model → Individual Units
├── Tour/Show
├── SeparationList
├── WorkList
└── Financial (básico)
    ├── Quote (orçamento)
    ├── Invoice (fatura)
    └── Payment
```

Tecnologias

- **Backend:** Flask, SQLAlchemy, Flask-Login
 - **Frontend:** Jinja2, TailwindCSS, JavaScript vanilla
 - **Database:** SQLite (tour_manager.db)
 - **Deploy:** Replit
 - **Versionamento:** GitHub
-

3. ARQUITETURA ATUAL

Estrutura de Arquivos

```
/project
├── main.py                # App principal + migrations
├── config.py              # Configurações
├── extensions.py          # db, login_manager, bcrypt
├── models/
│   ├── company.py
│   ├── user.py
│   ├── equipment.py
│   ├── tour.py
│   ├── financial.py      # Quote, Invoice, Payment
│   └── [outros...]
├── routes/
│   ├── auth.py
│   ├── equipment.py
│   ├── tour.py
│   ├── financial.py      # Rotas admin-only
│   ├── orcamento.py     # Orçamento rápido
│   └── [outros...]
├── services/
│   ├── payment_adapter.py # Asaas + Manual
│   └── nfse_adapter.py    # Focus NFe + Manual
├── templates/
│   ├── base.html         # Layout principal
│   ├── financial/
│   └── [outros...]
└── static/
```

Fluxo Atual de Orçamento

1. Orçamento Rápido (nome + tel)
 - └ Busca equipamentos (brand + model)
 - └ Gera PDF/WhatsApp
 2. Se aprovado: "Cliente aprovou!"
 - └ Pede valor sinal
 - └ Cria Lead (histórico)
 - └ Cria Cliente (financeiro)
 - └ Cadastra dados completos
 3. Após sinal:
 - └ Cria Tour/Evento
 - └ Aloca equipamentos (reserved → allocated)
 - └ Gera checklist separação
-

4. NOVOS MÓDULOS

4.1 Estrutura Geral

MÓDULOS NOVOS (independentes):

- └ EXPENSE (Despesas)
 - └ Contas Fixas/Variáveis
 - └ Provisionamento automático
 - └ Pagamento inteligente
- └ RH (Recursos Humanos)
 - └ Funcionários CLT
 - └ Freelancers
 - └ Folha de pagamento
 - └ Ranking performance
- └ LEADS (Comercial/CRM)
 - └ Funil de vendas
 - └ Score de intenção
 - └ Estratégias fechamento
 - └ Reativação orçamentos
- └ INTELLIGENCE (IA)
 - └ Assistente Financeiro
 - └ Assistente Comercial
 - └ Previsões e alertas

4.2 Princípio de Integração

IMPORTANTE: Módulos são INDEPENDENTES mas se COMUNICAM:

```
# Exemplo: Orçamento aprovado dispara automações
@orcamento_bp.route('/<int:id>/aprovar')
def aprovar():
    # ... código existente ...

# NOVO: Trigger automações
from services.automations import Automations
Automations.orcamento_aprovado(orcamento)
# Isso cria: Invoice, aloca Equipment, cria Tour
```

5. MÓDULO FINANCEIRO EXPANDIDO

5.1 Visão Geral

Expansão do módulo financeiro existente com: - Sistema de provisionamento (previsão de despesas/receitas) - Contas fixas com recorrência automática - Folha de pagamento CLT com cálculos automáticos - Pagamentos freelancer simplificados - Dashboards estratégicos com IA - Relatórios comparativos (YoY, MoM, WoW)

5.2 Models Novos

ExpenseCategory

Arquivo: models/expense.py

```
class ExpenseCategory(db.Model):
    """Categorias de despesas"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))

    nome = db.Column(db.String(100)) # 'Água', 'Luz', 'Internet'
    tipo = db.Column(db.String(20)) # 'fixa' ou 'variavel'
    icone = db.Column(db.String(10)) # emoji UTF-8
    cor = db.Column(db.String(20)) # hex color para UI

    # Padrão
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

Categorias Padrão: - 💧 Água (fixa) - ⚡ Energia (fixa) - 🌐 Internet (fixa) - 🏠 Aluguel (fixa) - ☎ Telefone (fixa) - 🛠 Manutenção (variável) - ⛽ Combustível (variável) - 📦 Material (variável)

AccountPayable (Conta a Pagar)

Arquivo: models/expense.py

```
class AccountPayable(db.Model):
    """Contas a pagar com PROVISIONAMENTO AUTOMÁTICO"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))
    category_id = db.Column(db.Integer, db.ForeignKey('expense_category.id'))

    # Dados básicos
    descricao = db.Column(db.String(200), nullable=False)
    fornecedor = db.Column(db.String(200))
    valor = db.Column(db.Numeric(10, 2), nullable=False)

    # Datas
    data_vencimento = db.Column(db.Date, nullable=False)
    data_pagamento = db.Column(db.Date) # quando foi pago

    # RECORRÊNCIA (contas fixas)
    recorrente = db.Column(db.Boolean, default=False)
    frequencia = db.Column(db.String(20)) # 'mensal', 'bimestral',
    'anual'
    dia_vencimento = db.Column(db.Integer) # 1-31
    provisionar_meses = db.Column(db.Integer, default=6)

    # PROVISIONAMENTO INTELIGENTE
    valor_medio = db.Column(db.Boolean, default=True) # usa média
    últimos 3 meses
    ultimo_valor_pago = db.Column(db.Numeric(10, 2)) # atualiza ao
    pagar

    # STATUS
    status = db.Column(db.String(20), default='provisionado')
    # Status possíveis:
    # - 'provisionado': gerado automaticamente, ainda não venceu
    # - 'a confirmar': vencimento próximo, aguarda confirmação valor
    # - 'confirmado': valor confirmado, pronto para pagar
    # - 'pago': pagamento realizado com comprovante
    # - 'atrasado': venceu e não foi pago
```

```

        tipo_provisionamento = db.Column(db.String(20)) # 'auto',
        'manual'

        # PAGAMENTO
        forma_pagamento = db.Column(db.String(20)) # 'pix', 'boleto',
        'debito'

        comprovante_url = db.Column(db.String(500))
        codigo_barras = db.Column(db.String(100)) # boleto
        pix_qr_code = db.Column(db.Text) # código PIX

        # RELACIONAMENTO (provisionamento)
        conta_origem_id = db.Column(db.Integer,
db.ForeignKey('account_payable.id'))
        # Se esta conta foi gerada automaticamente de outra recorrente

        observacoes = db.Column(db.Text)
        is_active = db.Column(db.Boolean, default=True)
        created_at = db.Column(db.DateTime, default=datetime.utcnow)
        created_by = db.Column(db.Integer, db.ForeignKey('user.id'))

```

Lógica de Provisionamento:

```

# Automação: Todo dia 1º do mês
def provisionar_contas_fixas():
    """Cria contas do mês atual + próximos N meses"""
    contas_recorrentes = AccountPayable.query.filter_by(
        recorrente=True,
        is_active=True
    ).all()

    for conta_master in contas_recorrentes:
        # Cria próximos 6 meses
        for mes in range(1, conta_master.provisionar_meses + 1):
            data_venc =
calcular_vencimento(conta_master.dia_vencimento, mes)

            # Verifica se já existe
            if not existe_conta_provisionada(conta_master,
data_venc):
                valor = (conta_master.ultimo_valor_pago if
conta_master.valor_medio
                        else conta_master.valor)

                criar_conta_provisionada(
                    conta_origem=conta_master,
                    data_vencimento=data_venc,
                    valor=valor,
                    status='provisionado'
                )

```

Atualização Inteligente:

```

# Quando usuário paga R$ 870 (provisão era R$ 850):
def pagar_conta(conta_id, valor_pago):
    conta = AccountPayable.query.get(conta_id)
    conta.status = 'pago'
    conta.data_pagamento = date.today()
    conta.valor = valor_pago # atualiza com valor real

# ATUALIZA PROVISÕES FUTURAS
if conta.conta_origem_id:
    conta_master =
AccountPayable.query.get(conta.conta_origem_id)
    conta_master.ultimo_valor_pago = valor_pago

# Atualiza próximas provisões ainda não pagas
AccountPayable.query.filter_by(
    conta_origem_id=conta_master.id,
    status='provisionado',
    data_vencimento > date.today()
).update({'valor': valor_pago})

db.session.commit()

```

Employee (Funcionário)

Arquivo: models/employee.py

```
class Employee(db.Model):
    """Funcionários CLT e Freelancers"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))

    # Tipo
    tipo = db.Column(db.String(20), nullable=False) # 'clt' ou
    'freelancer'

    # Dados pessoais
    nome = db.Column(db.String(200), nullable=False)
    cpf = db.Column(db.String(14))
    rg = db.Column(db.String(20))
    data_nascimento = db.Column(db.Date)

    # Contato
    telefone = db.Column(db.String(20))
    email = db.Column(db.String(120))

    # Profissional
    cargo = db.Column(db.String(100))
    data_admissao = db.Column(db.Date) # CLT only
    data_demissao = db.Column(db.Date)
    salario_base = db.Column(db.Numeric(10, 2)) # CLT only

    # Bancário
    banco = db.Column(db.String(100))
    agencia = db.Column(db.String(20))
    conta = db.Column(db.String(30))
    pix_key = db.Column(db.String(200))

    # Outros
    foto_url = db.Column(db.String(500))
    observacoes = db.Column(db.Text)

    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Relacionamentos
    folhas = db.relationship('Payroll', backref='funcionario',
                             lazy='dynamic')
    pagamentos_freelancer = db.relationship('FreelancerPayment',
                                             backref='funcionario', lazy='dynamic')
```

Payroll (Folha de Pagamento CLT)

Arquivo: models/employee.py

```
class Payroll(db.Model):
    """Folha de pagamento CLT com cálculos automáticos"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))
    employee_id = db.Column(db.Integer,
                             db.ForeignKey('employee.id'))

    # Período
    mes = db.Column(db.Integer, nullable=False) # 1-12
    ano = db.Column(db.Integer, nullable=False) # 2025

    # Valores
    salario_bruto = db.Column(db.Numeric(10, 2), nullable=False)

    # IMPOSTOS (calculados automaticamente)
    inss_percentual = db.Column(db.Numeric(5, 2)) # ex: 11.00
    inss_valor = db.Column(db.Numeric(10, 2))

    fgts_valor = db.Column(db.Numeric(10, 2)) # sempre 8%

    irrf_percentual = db.Column(db.Numeric(5, 2))
    irrf_valor = db.Column(db.Numeric(10, 2))
```

```

# BENEFÍCIOS
vale_transporte = db.Column(db.Numeric(10, 2), default=0)
vale_refeicao = db.Column(db.Numeric(10, 2), default=0)
plano_saude = db.Column(db.Numeric(10, 2), default=0)

# OUTROS
descontos_outros = db.Column(db.Numeric(10, 2), default=0)
bonus = db.Column(db.Numeric(10, 2), default=0)

# TOTAL
salario_liquido = db.Column(db.Numeric(10, 2)) # calculado

# STATUS
status = db.Column(db.String(20), default='rascunho')
# Status: 'rascunho' → 'aprovado' → 'pago'

data_pagamento = db.Column(db.Date)
comprovante_url = db.Column(db.String(500))
holerite_url = db.Column(db.String(500)) # PDF gerado

created_at = db.Column(db.DateTime, default=datetime.utcnow)
created_by = db.Column(db.Integer, db.ForeignKey('user.id'))
approved_by = db.Column(db.Integer, db.ForeignKey('user.id'))

```

Cálculos Automáticos:

```

def calcular_folha(employee_id, mes, ano):
    """Gera folha com cálculos automáticos de impostos"""
    emp = Employee.query.get(employee_id)
    salario_bruto = emp.salario_base

    # INSS (tabela 2025)
    if salario_bruto <= 1412:
        inss_perc = Decimal('7.5')
    elif salario_bruto <= 2666.68:
        inss_perc = Decimal('9.0')
    elif salario_bruto <= 4000.03:
        inss_perc = Decimal('12.0')
    else:
        inss_perc = Decimal('14.0')

    inss_valor = salario_bruto * inss_perc / 100

    # FGTS (sempre 8% sobre bruto, empresa paga)
    fgts_valor = salario_bruto * Decimal('0.08')

    # IRRF (tabela 2025 simplificada)
    base_irrf = salario_bruto - inss_valor
    if base_irrf <= 2259.20:
        irrf_perc = Decimal('0')
        irrf_valor = Decimal('0')
    elif base_irrf <= 2826.65:
        irrf_perc = Decimal('7.5')
        irrf_valor = base_irrf * irrf_perc / 100 - Decimal('169.44')
    elif base_irrf <= 3751.05:
        irrf_perc = Decimal('15.0')
        irrf_valor = base_irrf * irrf_perc / 100 - Decimal('381.44')
    elif base_irrf <= 4664.68:
        irrf_perc = Decimal('22.5')
        irrf_valor = base_irrf * irrf_perc / 100 - Decimal('662.77')
    else:
        irrf_perc = Decimal('27.5')
        irrf_valor = base_irrf * irrf_perc / 100 - Decimal('896.00')

    # LÍQUIDO
    salario_liquido = salario_bruto - inss_valor - irrf_valor

    payroll = Payroll(
        employee_id=employee_id,
        company_id=emp.company_id,
        mes=mes,
        ano=ano,
        salario_bruto=salario_bruto,
        inss_percentual=inss_perc,

```

```

        inss_valor=inss_valor,
        fgts_valor=fgts_valor,
        irrf_percentual=irrf_perc,
        irrf_valor=irrf_valor,
        salario_liquido=salario_liquido,
        status='rascunho'
    )

    return payroll

```

Geração Automática:

```

# Cronjob: Todo dia 25 do mês
def gerar_folhas_mes():
    """Gera folhas de todos funcionários CLT ativos"""
    mes_atual = date.today().month
    ano_atual = date.today().year

    funcionarios_clt = Employee.query.filter_by(
        tipo='clt',
        is_active=True
    ).all()

    for emp in funcionarios_clt:
        # Verifica se já existe
        if not Payroll.query.filter_by(
            employee_id=emp.id,
            mes=mes_atual,
            ano=ano_atual
        ).first():
            folha = calcular_folha(emp.id, mes_atual, ano_atual)
            db.session.add(folha)

    db.session.commit()
    # Notifica admin: "Folha de {mes} gerada, revisar antes de
aprovar"

```

FreelancerPayment

Arquivo: models/employee.py

```

class FreelancerPayment(db.Model):
    """Pagamentos simplificados para freelancers"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))
    employee_id = db.Column(db.Integer,
db.ForeignKey('employee.id'))

    # Serviço
    descricao_servico = db.Column(db.String(500), nullable=False)
    data_servico = db.Column(db.Date)

    # Valores
    valor_bruto = db.Column(db.Numeric(10, 2), nullable=False)
    retencao_ir_percentual = db.Column(db.Numeric(5, 2), default=0)
# opcional
    retencao_ir_valor = db.Column(db.Numeric(10, 2), default=0)
    valor_liquido = db.Column(db.Numeric(10, 2)) # calculado

    # Pagamento
    data_pagamento = db.Column(db.Date)
    forma_pagamento = db.Column(db.String(20)) # 'pix',
'transferencia', 'dinheiro'
    comprovante_url = db.Column(db.String(500))

    # Recibo
    recibo_gerado = db.Column(db.Boolean, default=False)
    recibo_url = db.Column(db.String(500)) # PDF auto-gerado

    # Status
    status = db.Column(db.String(20), default='pendente') #
'pendente', 'pago'

    observacoes = db.Column(db.Text)

```



```
created_at = db.Column(db.DateTime, default=datetime.utcnow)
created_by = db.Column(db.Integer, db.ForeignKey('user.id'))
```

Fluxo Simplificado:

1. Admin seleciona freelancer
 2. Digita: descrição + valor
 3. Sistema calcula líquido (se tiver retenção IR)
 4. Registra pagamento
 5. Gera recibo simples automaticamente (PDF)
-

6. MÓDULO COMERCIAL COM IA

6.1 Visão Geral

Sistema completo de CRM (Customer Relationship Management) com:

- Funil de vendas visual (kanban) - Score de intenção automatizado (0-100) - Assistente IA para estratégias de fechamento - Reativação inteligente de orçamentos recusados - Análise de equipamentos disponíveis - Histórico completo de interações

6.2 Models Novos

Lead

Arquivo: models/lead.py

```
class Lead(db.Model):
    """Lead no funil de vendas"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))

    # Cliente
    cliente_nome = db.Column(db.String(200), nullable=False)
    cliente_telefone = db.Column(db.String(20))
    cliente_email = db.Column(db.String(120))
    cliente_empresa = db.Column(db.String(200))
    cliente_cpf_cnpj = db.Column(db.String(18))

    # Origem
    origem = db.Column(db.String(50)) # 'instagram', 'site',
    'indicacao', 'whatsapp'
    origem_detalhes = db.Column(db.String(200)) # ex: "Post do dia
    15/12"

    # Funil
    status = db.Column(db.String(30), default='novo')
    # Status do funil:
    # 'novo' → 'contato' → 'orcamento' → 'negociacao' → 'ganho' /
    'perdido'

    # TEMPERATURA (calculada automaticamente pela IA)
    temperatura = db.Column(db.String(20), default='frio') #
    'frio', 'morno', 'quente'
    score_intencao = db.Column(db.Integer, default=30) # 0-100

    # Evento
    evento_tipo = db.Column(db.String(100)) # 'show', 'festival',
    'corporativo'
    evento_data = db.Column(db.Date)
    evento_local = db.Column(db.String(200))
    evento_publico_estimado = db.Column(db.Integer)

    # Comercial
    valor_estimado = db.Column(db.Numeric(10, 2))
    orcamento_id = db.Column(db.Integer, db.ForeignKey('quote.id'))
    # se converteu
    responsavel_id = db.Column(db.Integer, db.ForeignKey('user.id'))
```

```

        # Perda (se status='perdido')
        motivo_perda = db.Column(db.String(50)) # 'preco', 'prazo',
        'concorrencia', 'sem resposta'
        motivo_perda_detalhes = db.Column(db.Text)
        data_perda = db.Column(db.DateTime)

        # Ganho (se status='ganho')
        data_ganho = db.Column(db.DateTime)
        valor_fechado = db.Column(db.Numeric(10, 2))

        # REATIVACÃO (para leads perdidos)
        reativacao_tentativas = db.Column(db.Integer, default=0)
        ultima_reativacao = db.Column(db.DateTime)
        pode_reativar = db.Column(db.Boolean, default=True)

        # Datas
        data_ultimo_contato = db.Column(db.DateTime)
        created_at = db.Column(db.DateTime, default=datetime.utcnow)
        created_by = db.Column(db.Integer, db.ForeignKey('user.id'))

        observacoes = db.Column(db.Text)
        is_active = db.Column(db.Boolean, default=True)

        # Relacionamentos
        atividades = db.relationship('LeadActivity', backref='lead',
        lazy='dynamic', order_by='LeadActivity.created_at.desc()')

```

LeadActivity (Histórico)

Arquivo: models/lead.py

```

class LeadActivity(db.Model):
    """Histórico de atividades do lead"""
    id = db.Column(db.Integer, primary_key=True)
    lead_id = db.Column(db.Integer, db.ForeignKey('lead.id'))
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))

    tipo = db.Column(db.String(50), nullable=False)
    # Tipos:
    # 'ligacao', 'whatsapp', 'email', 'reuniao',
    'orcamento_enviado',
    # 'orcamento_visualizado', 'negociacao', 'status_mudou'

    titulo = db.Column(db.String(200))
    descricao = db.Column(db.Text)

    # Metadados
    duracao_minutos = db.Column(db.Integer) # para
    ligacoes/reuniões
    anexo_url = db.Column(db.String(500)) # print WhatsApp, email,
    etc

    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    created_by = db.Column(db.Integer, db.ForeignKey('user.id'))

```

6.3 Sistema de Score de Intenção (IA)

Arquivo: services/commercial_intelligence.py

```

class CommercialIntelligence:
    """Inteligência artificial para vendas"""

    @staticmethod
    def calcular_score_lead(lead_id):
        """
        Calcula score 0-100 baseado em múltiplos fatores
        Atualiza temperatura automaticamente
        """
        lead = Lead.query.get(lead_id)
        score = 0

        # FATOR 1: Tempo desde criação (até 20 pts)
        dias_desde_criacao = (datetime.utcnow() -

```

```

lead.created_at).days
    if dias_desde_criacao <= 3:
        score += 20
    elif dias_desde_criacao <= 7:
        score += 15
    elif dias_desde_criacao <= 14:
        score += 10
    else:
        score += 5

    # FATOR 2: Frequência de contato (até 25 pts)
    atividades_ultimos_7_dias = lead.atividades.filter(
        LeadActivity.created_at >= datetime.utcnow() -
timedelta(days=7)
    ).count()

    if atividades_ultimos_7_dias >= 3:
        score += 25
    elif atividades_ultimos_7_dias >= 2:
        score += 15
    elif atividades_ultimos_7_dias >= 1:
        score += 10

    # FATOR 3: Proximidade do evento (até 20 pts)
    if lead.evento_data:
        dias_ate_evento = (lead.evento_data - date.today()).days
        if 0 < dias_ate_evento <= 30:
            score += 20 # evento próximo = urgente!
        elif 30 < dias_ate_evento <= 60:
            score += 15
        elif 60 < dias_ate_evento <= 90:
            score += 10

    # FATOR 4: Valor estimado (até 15 pts)
    if lead.valor_estimado:
        if lead.valor_estimado >= 30000:
            score += 15
        elif lead.valor_estimado >= 15000:
            score += 10
        elif lead.valor_estimado >= 5000:
            score += 5

    # FATOR 5: Orçamento enviado (até 10 pts)
    orcamento_enviado =
lead.atividades.filter_by(tipo='orcamento_enviado').first()
    if orcamento_enviado:
        score += 10

    # FATOR 6: Dados completos (até 10 pts)
    dados_completos = sum([
        bool(lead.cliente_telefone),
        bool(lead.cliente_email),
        bool(lead.evento_data),
        bool(lead.valor_estimado)
    ])
    score += dados_completos * 2.5

    # AJUSTES NEGATIVOS
    # Sem contato há muito tempo
    if lead.data_ultimo_contato:
        dias_sem_contato = (datetime.utcnow() -
lead.data_ultimo_contato).days
        if dias_sem_contato > 7:
            score -= min(20, dias_sem_contato * 2) # máximo

-20pts

    # Limitar 0-100
    score = max(0, min(100, score))

    # ATUALIZAR TEMPERATURA
    if score >= 75:
        temperatura = 'quente'
    elif score >= 50:
        temperatura = 'morno'
    else:

```

```

        temperatura = 'frio'

    # Salvar
    lead.score_intencao = score
    lead.temperatura = temperatura
    db.session.commit()

    return {
        'score': score,
        'temperatura': temperatura,
        'fatores': {
            'tempo': dias_desde_criacao,
            'contatos': atividades_ultimos_7_dias,
            'dias_evento': dias_ate_evento if lead.evento_data
        }
    }
else None

```

6.4 Estratégias de Fechamento (IA)

Arquivo: services/commercial_intelligence.py

```

@staticmethod
def gerar_estrategias_fechamento(orcamento_id):
    """
    Analisa orçamento e retorna estratégias personalizadas
    Baseado em: equipamentos disponíveis, margem, histórico
    """

    from models.financial import Quote
    from models.equipment import Equipment

    orcamento = Quote.query.get(orcamento_id)
    estrategias = []

    # ESTRATÉGIA 1: DESCONTO PROGRESSIVO
    margem_atual = calcular_margem_orcamento(orcamento)

    for desconto_perc in [5, 10, 15]:
        nova_margem = margem_atual - desconto_perc

        if nova_margem >= 25: # margem mínima aceitável
            valor_com_desconto = orcamento.total * (1 -
desconto_perc/100)

            estrategias.append({
                'tipo': 'desconto_progressivo',
                'desconto_percentual': desconto_perc,
                'valor_original': float(orcamento.total),
                'valor_com_desconto': float(valor_com_desconto),
                'economia_cliente': float(orcamento.total -
valor_com_desconto),
                'margem_resultante': nova_margem,
                'viavel': True,
                'pitch': f"Revisamos os custos e conseguimos
oferecer {desconto_perc}% "
                        f"de desconto se fecharmos esta semana. "
                        f"De R$ {orcamento.total:,.2f} por R$
{valor_com_desconto:,.2f}",
                'prioridade': 'alta' if desconto_perc <= 10 else
'media'
            })

    # ESTRATÉGIA 2: EQUIPAMENTOS ALTERNATIVOS
    # Verifica se há equipamentos similares mais baratos disponíveis
    equipamentos_orcamento = orcamento.items # itens do orçamento

    for item in equipamentos_orcamento:
        # Busca alternativas
        alternativas = buscar_equipamentos_similares(
            tipo=item.equipment_type,
            data_evento=orcamento.lead.evento_data if orcamento.lead
else None,
            preco_max=item.unit_price * 0.8 # até 20% mais barato
        )

```

```

        if alternativas:
            economia_total = (item.unit_price - alternativas[0]
                               ['preco']) * item.quantity

            estrategias.append({
                'tipo': 'equipamento alternativo',
                'item_original': item.description,
                'alternativa': alternativas[0]['nome'],
                'economia': float(economia_total),
                'diferenca_qualidade': alternativas[0]
                ['qualidade_percent'], # ex: 85%
                'disponivel': alternativas[0]['disponivel'],
                'pitch': f"Temos {alternativas[0]['nome']}
disponível "
                                f"(80% da potência) por R$
{economia_total:,.2f} a menos",
                'prioridade': 'media'
            })

        # ESTRATÉGIA 3: SUBLOCAÇÃO
        # Se faltam equipamentos, busca parceiros
        equipamentos_faltando =
verificar_disponibilidade_total(orcamento)

        if equipamentos_faltando:
            parceiros =
buscar_parceiros_sublocacao(equipamentos_faltando)

            if parceiros:
                custo_sublocacao = sum(p['custo'] for p in parceiros)
                margem_com_sublocacao = margem_atual - (custo_sublocacao
                / orcamento.total * 100)

                if margem_com_sublocacao >= 30:
                    estrategias.append({
                        'tipo': 'sublocacao',
                        'equipamentos_faltando': equipamentos_faltando,
                        'parceiros': parceiros,
                        'custo_adicional': float(custo_sublocacao),
                        'margem_resultante': margem_com_sublocacao,
                        'viavel': True,
                        'pitch': f"Podemos fechar locando
{len(equipamentos_faltando)} itens "
                                f"de parceiros confiáveis mantendo o
prazo",
                        'prioridade': 'alta'
                    })

        # ESTRATÉGIA 4: URGÊNCIA TEMPORAL
        if orcamento.lead and orcamento.lead.evento_data:
            dias_ate_evento = (orcamento.lead.evento_data -
date.today()).days

            if dias_ate_evento <= 45: # evento próximo
                estrategias.append({
                    'tipo': 'urgencia_temporal',
                    'dias_ate_evento': dias_ate_evento,
                    'pitch': f"0 evento está em {dias_ate_evento} dias e
nossa agenda "
                                f"está se fechando. Podemos garantir os
equipamentos se "
                                f"fechamos até sexta-feira.",
                    'prioridade': 'alta' if dias_ate_evento <= 30 else
'media'
                })

        # ESTRATÉGIA 5: COMBO COM VALOR AGREGADO
        # Inclui serviços grátis
        servicos_gratis = [
            {'nome': 'Técnico durante evento', 'custo': 800,
'valor_percebido': 800},
            {'nome': 'Seguro equipamentos', 'custo': 200,
'valor_percebido': 400},
            {'nome': 'Transporte', 'custo': 300, 'valor_percebido': 500}

```

```

    ]

    custo_total_servicos = sum(s['custo'] for s in servicos_gratis)
    valor_percebido_total = sum(s['valor_percebido'] for s in
servicos_gratis)
    margem_com_servicos = margem_atual - (custo_total_servicos /
orcamento.total * 100)

    if margem_com_servicos >= 28:
        estrategias.append({
            'tipo': 'valor_agregado',
            'servicos': servicos_gratis,
            'custo_real': float(custo_total_servicos),
            'valor_percebido': float(valor_percebido_total),
            'margem_resultante': margem_com_servicos,
            'pitch': f"Incluimos técnico, seguro e transporte sem
custo adicional "
                    f"(valor de R$ {valor_percebido_total:,.2f})",
            'prioridade': 'alta'
        })

    # Ordenar por prioridade
    estrategias.sort(key=lambda x: 0 if x['prioridade']=='alta' else
1)

    return estrategias

```

6.5 Reativação de Leads Perdidos

Arquivo: services/commercial_intelligence.py

```

@staticmethod
def analisar_oportunidade_reativacao(lead_id):
    """
    Verifica se vale a pena tentar reativar um lead perdido
    Retorna estratégias específicas
    """
    lead = Lead.query.get(lead_id)

    if lead.status != 'perdido':
        return {'reativar': False, 'motivo': 'Lead não está
perdido'}

    if not lead.pode_reativar:
        return {'reativar': False, 'motivo': 'Bloqueado para
reativação'}

    if lead.reativacao_tentativas >= 3:
        return {'reativar': False, 'motivo': 'Máximo de tentativas
atingido'}

    # SINAIS POSITIVOS
    sinais_positivos = []
    score_reativacao = 0

    # 1. Evento ainda não aconteceu
    if lead.evento_data and lead.evento_data > date.today():
        dias_ate_evento = (lead.evento_data - date.today()).days
        if dias_ate_evento >= 15: # tempo suficiente
            sinais_positivos.append('Evento ainda não aconteceu')
            score_reativacao += 30

    # 2. Não fechou com concorrente (checagem manual ou via redes
sociais)
    # Isso seria implementado com scraping/API Instagram
    # Por ora, assumir verdadeiro se < 30 dias
    dias_desde_perda = (datetime.utcnow() - lead.data_perda).days
    if dias_desde_perda <= 30:
        sinais_positivos.append('Provavelmente não fechou com
concorrente ainda')
        score_reativacao += 20

    # 3. Motivo da perda é negociável

```

```

        if lead.motivo_perda in ['preco', 'prazo']:
            sinais_positivos.append(f'Motivo negociável:
{lead.motivo_perda}')
            score_reativacao += 25

# 4. Equipamentos disponíveis
if lead.orcamento_id:
    orcamento = Quote.query.get(lead.orcamento_id)
    if verificar_disponibilidade_total(orcamento):
        sinais_positivos.append('Equipamentos disponíveis')
        score_reativacao += 15

# 5. Valor alto (vale o esforço)
if lead.valor_estimado and lead.valor_estimado >= 10000:
    sinais_positivos.append(f'Valor alto: R$
{lead.valor_estimado:,.2f}')
    score_reativacao += 10

# DECISÃO
if score_reativacao < 40:
    return {
        'reativar': False,
        'score': score_reativacao,
        'motivo': 'Score muito baixo, focar em novos leads'
    }

# GERAR ESTRATÉGIAS DE RECONQUISTA
estrategias = []

if lead.motivo_perda == 'preco':
    # Proposta econômica
    if lead.orcamento_id:
        orcamento_original = Quote.query.get(lead.orcamento_id)

        estrategias.append({
            'nome': 'Proposta Econômica',
            'descricao': 'Pacote com menos itens não-
essenciais',

            'reducao_estimada': '15-20%',
            'acao': 'Criar orçamento revisado removendo itens
secundários'

        })

        estrategias.append({
            'nome': 'Desconto Direto',
            'descricao': 'Oferecer 10% desconto se fechar em
48h',

            'reducao_estimada': '10%',
            'acao': 'Enviar proposta urgente com deadline'

        })

    elif lead.motivo_perda == 'prazo':
        estrategias.append({
            'nome': 'Sublocação Express',
            'descricao': 'Garantir equipamentos via parceiros',
            'acao': 'Contatar rede de parceiros imediatamente'

        })

# Estratégia genérica: Janela de Oportunidade
if lead.evento_data:
    fim_semana_evento = f"{lead.evento_data.day}-
{lead.evento_data.day+2}/{lead.evento_data.month}"

    # Verifica se temos eventos naquela data
    eventos_mesma_data = Tour.query.filter(
        Tour.start_date == lead.evento_data,
        Tour.company_id == lead.company_id
    ).count()

    if eventos_mesma_data == 0:
        estrategias.append({
            'nome': 'Janela de Oportunidade',
            'descricao': f'Fim de semana {fim_semana_evento} sem
eventos agendados',

            'acao': 'Oferecer margem reduzida (25%) - melhor que

```

```

equipamento parado'
    })

    return {
        'reativar': True,
        'score': score_reativacao,
        'sinais_positivos': sinais_positivos,
        'estrategias': estrategias,
        'urgencia': 'alta' if dias_ate_evento <= 30 else 'media'
    }

```

7. MÓDULO RH

(Já detalhado na seção 5.2 - Employee, Payroll, FreelancerPayment)

7.1 Dashboards e Rankings

Arquivo: services/rh_analytics.py

```

class RHPAnalytics:
    """Analytics e rankings de RH"""

    @staticmethod
    def ranking_funcionarios(company_id, mes=None, ano=None):
        """
        Gera ranking de performance dos funcionários
        Baseado em: tours, equipamentos gerenciados, checklists OK
        """
        if not mes:
            mes = date.today().month
        if not ano:
            ano = date.today().year

        funcionarios = Employee.query.filter_by(
            company_id=company_id,
            tipo='clt',
            is_active=True
        ).all()

        ranking = []

        for func in funcionarios:
            # 1. Tours participadas
            tours = TourAccess.query.filter_by(
                user_id=func.user_id # assumindo link Employee-
>User
            ).filter(
                func(Tour.start_date).between(
                    date(ano, mes, 1),
                    date(ano, mes, 31)
                )
            ).count()

            # 2. Equipamentos sob responsabilidade
            equipamentos = Equipment.query.filter_by(
                responsavel_id=func.user_id
            ).count()

            # 3. Checklists 100% OK (separations sem problemas)
            checklists_ok = SeparationList.query.filter_by(
                created_by=func.user_id,
                status='completo'
            ).filter(
                func(SeparationList.created_at).between(
                    datetime(ano, mes, 1),
                    datetime(ano, mes, 31, 23, 59)
                )
            ).count()

            # 4. Performance score (0-100)

```



```

        score = min(100, (
            (tours * 10) +
            (equipamentos * 2) +
            (checklists_ok * 15)
        ))

        ranking.append({
            'funcionario': func,
            'tours': tours,
            'equipamentos': equipamentos,
            'checklists_ok': checklists_ok,
            'performance_score': score
        })

    # Ordenar por score
    ranking.sort(key=lambda x: x['performance_score'],
reverse=True)

    return ranking

```

8. MÓDULO MANUTENÇÃO

8.1 Expansão do Model Existente

Arquivo: models/maintenance.py (EXPANDIR)

```

# ADICIONAR aos campos existentes:

custo_pecas = db.Column(db.Numeric(10, 2), default=0)
custo_mao_obra = db.Column(db.Numeric(10, 2), default=0)
custo_total = db.Column(db.Numeric(10, 2)) # auto-calculado

tempo_parado_dias = db.Column(db.Integer) # dias que equipamento
ficou parado
tecnico_responsavel_id = db.Column(db.Integer,
db.ForeignKey('employee.id'))

prioridade = db.Column(db.String(20), default='media') # baixa,
media, alta, urgente
recorrencia = db.Column(db.Boolean, default=False) # problema
repetido?

# Manutenção preventiva
tipo_manutencao = db.Column(db.String(20)) # 'corretiva',
'preventiva', 'calibracao'

```

8.2 Manutenção Preventiva

NOVO Model: MaintenanceSchedule

```

class MaintenanceSchedule(db.Model):
    """Agendamento de manutenções preventivas"""
    id = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))
    equipment_id = db.Column(db.Integer,
db.ForeignKey('equipment.id'))

    tipo = db.Column(db.String(30)) # 'preventiva', 'calibracao'
    frequencia_dias = db.Column(db.Integer) # ex: 90, 180, 365

    ultima_manutencao = db.Column(db.Date)
    proxima_manutencao = db.Column(db.Date) # auto-calculado

    status = db.Column(db.String(20), default='agendado') #
agendado, atrasado, concluido

    observacoes = db.Column(db.Text)
    is_active = db.Column(db.Boolean, default=True)

```

8.3 Analytics de Manutenção

Arquivo: services/maintenance_analytics.py

```
class MaintenanceAnalytics:
    """Analytics de manutenção"""

    @staticmethod
    def equipamentos_criticos(company_id, ano=None):
        """
        Lista equipamentos com mais manutenções e maiores custos
        """
        if not ano:
            ano = date.today().year

        # Manutenções por equipamento
        result = db.session.query(
            Equipment.id,
            Equipment.code,
            Equipment.brand,
            Equipment.model,
            func.count(Maintenance.id).label('total_manutencoes'),

func.sum(Maintenance.custo_total).label('custo_total_ano')
        ).join(Maintenance).filter(
            Equipment.company_id == company_id,
            func(Maintenance.created_at).year == ano
        ).group_by(Equipment.id).all()

        criticos = []
        for eq in result:
            # Flag se equipamento é crítico
            is_critico = (
                eq.total_manutencoes >= 5 or # 5+ manutenções/ano
                eq.custo_total_ano >= eq.equipment.valor * 0.5 #
custo > 50% valor
            )

            if is_critico:
                criticos.append({
                    'equipment': eq.equipment,
                    'total_manutencoes': eq.total_manutencoes,
                    'custo_total': eq.custo_total_ano,
                    'recomendacao': 'SUBSTITUIR' if
eq.custo_total_ano >= eq.equipment.valor * 0.7 else 'MONITORAR'
                })

        return criticos
```

9. INTELIGÊNCIA ARTIFICIAL

9.1 Assistente Financeiro

Arquivo: services/financial_intelligence.py

```
class FinancialIntelligence:
    """IA para decisões financeiras estratégicas"""

    @staticmethod
    def analise_fluxo_caixa(company_id, dias=15):
        """
        Prevê fluxo de caixa dia-a-dia nos próximos N dias
        Retorna timeline com saldo projetado
        """
        hoje = date.today()

        # Saldo inicial (simplificado - assumir campo no Company ou
calcular)

        saldo_atual = obter_saldo_atual(company_id)
```

```

        timeline = []
        saldo_acumulado = saldo_atual

        for dia_offset in range(dias + 1):
            data = hoje + timedelta(days=dia_offset)

            # ENTRADAS previstas
            entradas =
db.session.query(func.sum(Invoice.total)).filter(
                Invoice.company_id == company_id,
                Invoice.due_date == data,
                Invoice.status.in_(['pending', 'partial'])
            ).scalar() or 0

            # Ajustar por probabilidade (se pending há muito tempo,
reduzir)

            # Simplificado aqui

            # SAÍDAS previstas
            saidas =
db.session.query(func.sum(AccountPayable.valor)).filter(
                AccountPayable.company_id == company_id,
                AccountPayable.data_vencimento == data,
                AccountPayable.status.in_(['provisionado',
'a_confirmar']))
            ).scalar() or 0

            # Saldo do dia
            saldo_acumulado += (entradas - saidas)

            # Classificar risco
            if saldo_acumulado < 5000:
                risco = 'ALTO'
            elif saldo_acumulado < 15000:
                risco = 'MEDIO'
            else:
                risco = 'BAIXO'

            timeline.append({
                'data': data,
                'entradas': float(entradas),
                'saidas': float(saidas),
                'saldo': float(saldo_acumulado),
                'risco': risco
            })

        return timeline

    @staticmethod
    def estrategia_pagamentos(company_id):
        """
        Recomenda QUANDO pagar cada conta para otimizar fluxo de
caixa

        Considera: descontos, juros, necessidade de caixa
        """
        hoje = date.today()

        # Obter todas contas a pagar próximos 30 dias
        contas = AccountPayable.query.filter(
            AccountPayable.company_id == company_id,
            AccountPayable.status.in_(['provisionado',
'a_confirmar']),
            AccountPayable.data_vencimento.between(hoje, hoje +
timedelta(days=30))
        ).order_by(AccountPayable.data_vencimento).all()

        # Análise de fluxo
        fluxo =
FinancialIntelligence.analise_fluxo_caixa(company_id, dias=30)

        recomendacoes = {
            'pagar_ agora': [],
            'agendar': [],
            'negociar_prazo': []
        }

```

```

        for conta in contas:
            # Verifica desconto antecipado (simulado - poderia vir
do fornecedor)
            desconto_antecipado =
verificar_desconto_antecipado(conta)

            # Verifica saldo no dia do vencimento
            saldo_no_vencimento = next(
                (f['saldo'] for f in fluxo if f['data'] ==
conta.data_vencimento),
                None
            )

            dias_ate_vencimento = (conta.data_vencimento -
hoje).days

            # LÓGICA DE DECISÃO
            if desconto_antecipado and desconto_antecipado['valor']
> 20:

                # Vale a pena pagar agora
                recomendacoes['pagar_agora'].append({
                    'conta': conta,
                    'motivo': 'Desconto antecipado',
                    'economia': desconto_antecipado['valor'],
                    'prioridade': 'alta'
                })

            elif saldo_no_vencimento and saldo_no_vencimento <
conta.valor:

                # Saldo insuficiente no vencimento
                recomendacoes['negociar_prazo'].append({
                    'conta': conta,
                    'motivo': 'Saldo insuficiente previsto',
                    'saldo_previsto': saldo_no_vencimento,
                    'sugestao_nova_data': conta.data_vencimento +
timedelta(days=10),
                    'prioridade': 'urgente'
                })

            else:

                # Pagar próximo ao vencimento
                recomendacoes['agendar'].append({
                    'conta': conta,
                    'data_sugerida': conta.data_vencimento -
timedelta(days=1),
                    'motivo': 'Manter caixa disponível',
                    'prioridade': 'normal'
                })

            # Calcular economia total
            economia_total = sum(
                c['economia'] for c in recomendacoes['pagar_agora']
            )

            return {
                'recomendacoes': recomendacoes,
                'economia_potencial': economia_total,
                'resumo': {
                    'pagar_agora': len(recomendacoes['pagar_agora']),
                    'agendar': len(recomendacoes['agendar']),
                    'negociar': len(recomendacoes['negociar_prazo'])
                }
            }
    }

```

9.2 Previsões e Alertas

Arquivo: services/predictive_analytics.py

```

class PredictiveAnalytics:
    """Análises preditivas e alertas inteligentes"""

    @staticmethod

```

```

def previsao_receita_mes(company_id, meses_frente=3):
    """
    Prevê receita dos próximos meses baseado em:
    - Média móvel últimos 6 meses
    - Sazonalidade
    - Orçamentos em negociação
    """
    # Histórico últimos 12 meses
    historico = db.session.query(
        func(Invoice.created_at).month.label('mes'),
        func.sum(Invoice.total).label('total')
    ).filter(
        Invoice.company_id == company_id,
        Invoice.status == 'paid',
        Invoice.created_at >= datetime.utcnow() -
timedelta(days=365)
    ).group_by(func(Invoice.created_at).month).all()

    # Média móvel simples
    media_mensal = sum(h.total for h in historico) /
len(historico)

    # Orçamentos quentes (70%+ probabilidade fechar)
    orcamentos_quentes = Quote.query.join(Lead).filter(
        Quote.company_id == company_id,
        Quote.status == 'sent',
        Lead.score_intencao >= 70
    ).all()

    valor_orcamentos_quentes = sum(o.total for o in
orcamentos_quentes)

    previsoes = []
    for mes_offset in range(1, meses_frente + 1):
        data_futura = date.today() + timedelta(days=30 *
mes_offset)

        # Base: média histórica
        receita_prevista = media_mensal

        # Adiciona orçamentos quentes com probabilidade ajustada
        receita_prevista += valor_orcamentos_quentes * 0.4 #
40% dos quentes

        previsoes.append({
            'mes': data_futura.month,
            'ano': data_futura.year,
            'receita_prevista': float(receita_prevista),
            'confianca': 'media' # poderia ser calculada
        })

    return previsoes

@staticmethod
def detectar_anomalias(company_id):
    """
    Detecta anomalias e gera alertas
    """
    alertas = []

    # 1. Despesa muito acima da média
    mes_atual = date.today().month
    despesa_mes_atual =
db.session.query(func.sum(AccountPayable.valor)).filter(
        AccountPayable.company_id == company_id,
        func(AccountPayable.created_at).month == mes_atual,
        AccountPayable.status == 'pago'
    ).scalar() or 0

    # Média últimos 3 meses
    despesa_media =
db.session.query(func.avg(func.sum(AccountPayable.valor))).filter(
        AccountPayable.company_id == company_id,
        AccountPayable.status == 'pago',
        AccountPayable.created_at >= datetime.utcnow() -

```

```

timedelta(days=90)
        ).group_by(func(AccountPayable.created_at).month).scalar()
    or 0

    if despesa_mes_atual > despesa_media * 1.3: # 30% acima
        alertas.append({
            'tipo': 'despesa_alta',
            'gravidade': 'media',
            'mensagem': f'Despesas
{((despesa_mes_atual/despesa_media - 1) * 100):.0f}% acima da
média',
            'acao_sugerida': 'Revisar despesas variáveis'
        })

    # 2. Margem abaixo da meta
    # ... implementar similar

    # 3. Equipamentos ociosos
    equipamentos_ociosos = Equipment.query.filter(
        Equipment.company_id == company_id,
        Equipment.status == 'available',
        ~Equipment.id.in_(
            db.session.query(TourEquipment.equipment_id).filter(
                TourEquipment.created_at >= datetime.utcnow() -
timedelta(days=90)
            )
        )
    ).count()

    if equipamentos_ociosos > 10:
        alertas.append({
            'tipo': 'equipamentos_ociosos',
            'gravidade': 'baixa',
            'mensagem': f'{equipamentos_ociosos} equipamentos
sem uso há 90+ dias',
            'acao_sugerida': 'Considerar promoção ou venda'
        })

    return alertas

```

10. AUTOMAÇÕES

10.1 Sistema de Cronjobs

Arquivo: jobs/scheduler.py (NOVO)

```

from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
from services.automations import Automations

scheduler = BackgroundScheduler()

# Dia 1º de cada mês às 00:01 - Provisionar contas
scheduler.add_job(
    Automations.provisionar_contas_fixas_mes,
    CronTrigger(day=1, hour=0, minute=1),
    id='provisionar_contas'
)

# Dia 25 de cada mês às 08:00 - Gerar folhas CLT
scheduler.add_job(
    Automations.gerar_folhas_clt_mes,
    CronTrigger(day=25, hour=8, minute=0),
    id='gerar_folhas'
)

# Todo dia às 09:00 - Alertas inteligentes
scheduler.add_job(
    Automations.enviar_alertas_diarios,
    CronTrigger(hour=9, minute=0),

```

```

        id='alertas_diarios'
    )

    # Todo dia às 10:00 - Atualizar scores de leads
    scheduler.add_job(
        Automations.atualizar_scores_leads,
        CronTrigger(hour=10, minute=0),
        id='atualizar_scores'
    )

    # Segunda-feira 09:00 - Relatório semanal
    scheduler.add_job(
        Automations.enviar_relatorio_semanal,
        CronTrigger(day_of_week='mon', hour=9, minute=0),
        id='relatorio_semanal'
    )

def start_scheduler():
    """Inicia o scheduler"""
    scheduler.start()
    print("👉 Scheduler iniciado!")

def shutdown_scheduler():
    """Para o scheduler"""
    scheduler.shutdown()

```

10.2 Automações Principais

Arquivo: services/automations.py (NOVO)

```

class Automations:
    """Automações do sistema"""

    @staticmethod
    def provisionar_contas_fixas_meses(
        meses
    ):
        """
        Executa dia 1º: cria contas fixas do mês atual + próximos N
        meses

        """
        from models.expense import AccountPayable

        companies = Company.query.filter_by(is_active=True).all()

        for company in companies:
            contas_master = AccountPayable.query.filter_by(
                company_id=company.id,
                recorrente=True,
                is_active=True
            ).all()

            for master in contas_master:
                # Cria próximos 6 meses
                for offset_mes in range(1, master.provisionar_meses
                    + 1):
                    # ... lógica de criação ...
                    pass

            print(f"👉 Provisionamento concluído para {len(companies)}
empresas")

    @staticmethod
    def gerar_folhas_clt_mes():
        """
        Executa dia 25: gera folhas de todos CLT ativos
        """
        from models.employee import Employee, Payroll
        from services.notifications import enviar_notificacao

        mes_atual = date.today().month
        ano_atual = date.today().year

        funcionarios = Employee.query.filter_by(
            tipo='clt',

```

```

        is_active=True
    ).all()

    for func in funcionarios:
        # Verifica se já existe
        existe = Payroll.query.filter_by(
            employee_id=func.id,
            mes=mes_atual,
            ano=ano_atual
        ).first()

        if not existe:
            folha = calcular_folha(func.id, mes_atual,
ano_atual)

            db.session.add(folha)

    db.session.commit()

    # Notificar admin
    admins = User.query.filter_by(role='admin').all()
    for admin in admins:
        enviar_notificacao(
            user_id=admin.id,
            tipo='folha_gerada',
            mensagem=f'Folha de {mes_atual}/{ano_atual} gerada.
Revisar antes de aprovar.'
        )

    @staticmethod
    def enviar_alertas_diarios():
        """
        Todo dia 09:00: envia alertas personalizados
        """

        from services.financial_intelligence import
FinancialIntelligence
        from services.commercial_intelligence import
CommercialIntelligence

        companies = Company.query.filter_by(is_active=True).all()

        for company in companies:
            alertas = []

            # 1. Contas vencendo em 2 dias
            contas_urgentes = AccountPayable.query.filter(
                AccountPayable.company_id == company.id,
                AccountPayable.data_vencimento == date.today() +
timedelta(days=2),
                AccountPayable.status != 'pago'
            ).all()

            if contas_urgentes:
                alertas.append({
                    'tipo': 'conta_vencendo',
                    'mensagem': f'{len(contas_urgentes)} conta(s)
vencem em 2 dias',
                    'prioridade': 'alta'
                })

            # 2. Leads esfriando
            leads_esfriando = Lead.query.filter(
                Lead.company_id == company.id,
                Lead.status.in_(['contato', 'orcamento',
'negociacao']),
                Lead.data_ultimo_contato < datetime.utcnow() -
timedelta(days=5)
            ).all()

            if leads_esfriando:
                alertas.append({
                    'tipo': 'leads_esfriando',
                    'mensagem': f'{len(leads_esfriando)} leads sem
contato há 5+ dias',
                    'prioridade': 'media'
                })

```



```

        # 3. Saldo baixo previsto
        fluxo =
FinancialIntelligence.analise_fluxo_caixa(company.id, dias=7)
        saldo_minimo = min(f['saldo'] for f in fluxo)

        if saldo_minimo < 10000:
            alertas.append({
                'tipo': 'saldo_baixo',
                'mensagem': f'Saldo mínimo previsto (7 dias): R$
{saldo_minimo:,.2f}',
                'prioridade': 'alta'
            })

        # Enviar para admins
        # ... implementar envio ...

    @staticmethod
    def atualizar_scores_leads():
        """
        Atualiza score de todos leads ativos
        """
        from services.commercial_intelligence import
CommercialIntelligence

        leads_ativos = Lead.query.filter(
            Lead.status.in_(['novo', 'contato', 'orcamento',
'negociacao']))
        ).all()

        for lead in leads_ativos:
            CommercialIntelligence.calcular_score_lead(lead.id)

        print(f"👉 Scores atualizados: {len(leads_ativos)} leads")

    @staticmethod
    def orcamento_aprovado(orcamento_id):
        """
        Dispara quando orçamento é aprovado
        Cria: Invoice, aloca Equipment, cria Tour
        """
        from models.financial import Quote, Invoice
        from models.tour import Tour

        orcamento = Quote.query.get(orcamento_id)

        # 1. Criar Invoice
        invoice = Invoice(
            code=generate_code('FAT'),
            company_id=orcamento.company_id,
            client_name=orcamento.client_name,
            client_email=orcamento.client_email,
            client_document=orcamento.client_document,
            description=f'Ref: {orcamento.code}',
            subtotal=orcamento.total,
            total=orcamento.total,
            due_date=date.today() + timedelta(days=7),
            status='pending',
            created_by=orcamento.created_by
        )
        db.session.add(invoice)

        # 2. Alocar equipamentos (status: available → reserved)
        # ... lógica de alocação ...

        # 3. Criar Tour se tiver data de evento
        if orcamento.lead and orcamento.lead.evento_data:
            tour = Tour(
                name=orcamento.lead.evento_tipo or 'Tour',
                company_id=orcamento.company_id,
                start_date=orcamento.lead.evento_data,
                end_date=orcamento.lead.evento_data,
                client_name=orcamento.client_name,
                status='confirmed',
                created_by=orcamento.created_by
            )
            db.session.add(tour)

```

```

    )
    db.session.add(tour)

    db.session.commit()

    # 4. Gerar PIX (via payment_adapter.py existente)
    # ... chamada ao Asaas ...

```

11. INTERFACE E UX

11.1 Princípios de Design

Mobile-First: - Botões grandes (min 44x44px) - Touch-friendly - Navegação thumb-friendly - Mínimo scroll

Cores e Identidade:

```

:root {
  /* Cores principais */
  --primary: #f59e0b; /* Amber 500 */
  --primary-dark: #d97706; /* Amber 600 */
  --success: #10b981; /* Green 500 */
  --danger: #ef4444; /* Red 500 */
  --warning: #fbbf24; /* Yellow 400 */
  --info: #3b82f6; /* Blue 500 */

  /* Backgrounds */
  --bg-dark: #111827; /* Gray 900 */
  --bg-card: #1f2937; /* Gray 800 */
  --bg-hover: #374151; /* Gray 700 */

  /* Text */
  --text-primary: #ffffff;
  --text-secondary: #9ca3af; /* Gray 400 */
}

```

11.2 Componentes Reutilizáveis

templates/components/kpi_card.html:

```

<!-- KPI Card -->
<div class="bg-gradient-to-br from-{{cor}}-900 to-{{cor}}-800 p-4 rounded-lg shadow-lg">
  <div class="text-{{cor}}-300 text-sm mb-1">{{titulo}}</div>
  <div class="text-white text-2xl font-bold mb-2">{{valor}}</div>
  <div class="text-{{cor}}-400 text-xs">{{subtitulo}}</div>
</div>

```

templates/components/status_badge.html:

```

<!-- Badge de Status -->
<span class="px-3 py-1 rounded-full text-xs font-bold
  {% if status == 'pago' %}bg-green-900 text-green-300
  {% elif status == 'pendente' %}bg-yellow-900 text-yellow-300
  {% elif status == 'atrasado' %}bg-red-900 text-red-300
  {% else %}bg-gray-700 text-gray-300{% endif %}">
  {{ status|upper }}
</span>

```

11.3 Estrutura de Templates

```

templates/
├── base.html (já existe - reutilizar)
├── components/
│   ├── kpi_card.html
│   ├── status_badge.html
│   ├── timeline.html
│   └── funil.html

```

- └─ grafico_linha.html
- └─ expense/
 - └─ index.html (dashboard despesas)
 - └─ contas_pagar.html (lista + timeline)
 - └─ pagar_conta.html (form + estratégias IA)
 - └─ folha_clt.html
 - └─ freelancers.html
- └─ rh/
 - └─ dashboard.html (ranking)
 - └─ funcionarios.html (lista)
 - └─ form_clt.html
 - └─ form_freelancer.html
- └─ leads/
 - └─ kanban.html (funil visual)
 - └─ form.html
 - └─ view.html (detalhes + estratégias)
 - └─ reativar.html
- └─ financeiro/ (expandir existente)
 - └─ dashboard_estrategico.html
 - └─ previsao_fluxo.html
 - └─ dre.html
 - └─ comparativo.html

11.4 JavaScript Modular

static/js/modules/charts.js:

```
// Gráficos reutilizáveis com Chart.js
export function criarGraficoLinha(canvasId, dados, opcoes) {
  const ctx = document.getElementById(canvasId).getContext('2d');
  return new Chart(ctx, {
    type: 'line',
    data: dados,
    options: {
      responsive: true,
      maintainAspectRatio: false,
      ...opcoes
    }
  });
}

export function criarGraficoPizza(canvasId, dados, opcoes) {
  // ... similar ...
}
```

static/js/modules/api.js:

```
// Chamadas API centralizadas
export async function getFluxoCaixa(dias = 15) {
  const response = await fetch(`/api/financeiro/fluxo-caixa?
dias=${dias}`);
  return await response.json();
}

export async function getEstrategiasPagamento() {
  const response = await fetch('/api/financeiro/estrategias-
pagamento');
  return await response.json();
}
```

12. PLANO DE IMPLEMENTAÇÃO

12.1 Fases

FASE 1: FUNDAÇÃO (4-5 dias)

Objetivo: Criar estrutura base sem quebrar nada

Dia 1-2: Models - Criar models/expense.py completo - Criar models/employee.py completo - Criar models/lead.py completo - Expandir models/maintenance.py - Adicionar migrations em main.py - Testar criação de tabelas

Dia 3-4: Services IA - Criar services/financial_intelligence.py - Criar services/commercial_intelligence.py - Criar services/predictive_analytics.py - Criar services/automations.py - Testes unitários

Dia 5: Cronjobs - Criar jobs/scheduler.py - Integrar no main.py - Testar execução

Validação: Rodar sistema, verificar tabelas criadas, cronjobs funcionando

FASE 2: BACKEND ROUTES (5-6 dias)

Dia 6-7: Blueprint Expense - Criar routes/expense.py - Rotas: index, contas_pagar, pagar_conta, folha_clt, freelancers - Integração com FinancialIntelligence - Registrar no main.py

Dia 8-9: Blueprint RH - Criar routes/rh.py - Rotas: dashboard, funcionarios, novo_clt, novo_freelancer - Analytics e rankings

Dia 10-11: Blueprint Leads - Criar routes/leads.py - Rotas: kanban, form, view, estrategias, reativar - Integração com CommercialIntelligence

Validação: Acessar cada rota, verificar JSON responses, nenhum erro 500

FASE 3: FRONTEND (6-7 dias)

Dia 12-13: Templates Expense - Criar todos templates expense/ - Componentes reutilizáveis - JavaScript para gráficos

Dia 14-15: Templates RH + Leads - Templates rh/ - Templates leads/ (kanban com drag-drop) - Funil visual

Dia 16-17: Templates Financial expandido - Dashboard estratégico - Previsão fluxo - DRE - Comparativos temporais

Dia 18: Componentes e Polish - Componentes globais - Responsividade mobile - UX polish

Validação: Navegar todas telas mobile + desktop, sem bugs visuais

FASE 4: INTEGRAÇÕES E TESTES (3-4 dias)

Dia 19-20: Integrações - Conectar orçamento aprovado → automações - Conta paga → atualiza provisão - Lead convertido → cria tour - Testes end-to-end

Dia 21: Dados Seed - Popular categorias padrão - Criar dados de exemplo - Documentação

Dia 22: Testes Finais - Teste completo fluxo orçamento - Teste provisionamento - Teste IA recomendações - Correções de bugs

Validação: Sistema 100% funcional, sem quebras

12.2 Checklist de Validação

Financeiro ✓

- ☐ Contas fixas geram provisionamento automático
- ☐ Pagamento atualiza provisões futuras
- ☐ Folha CLT calcula impostos corretamente

- ☐ Freelancers geram recibo automaticamente
- ☐ Dashboard mostra KPIs corretos
- ☐ Fluxo de caixa prevê próximos 15 dias
- ☐ Estratégias de pagamento aparecem
- ☐ DRE gera corretamente

Comercial ✓

- ☐ Leads aparecem no kanban
- ☐ Score calcula automaticamente
- ☐ Temperatura atualiza (frio/morno/quente)
- ☐ Estratégias de fechamento aparecem
- ☐ Reativação detecta oportunidades
- ☐ Funil mostra conversão

RH ✓

- ☐ Funcionários listam corretamente
- ☐ Ranking calcula performance
- ☐ Folha gera dia 25 automaticamente
- ☐ Freelancers registram pagamento rápido

Automações ✓

- ☐ Cronjob dia 1º cria contas
- ☐ Cronjob dia 25 gera folhas
- ☐ Alertas diários funcionam
- ☐ Scores atualizam automaticamente

Integrações ✓

- ☐ Orçamento aprovado cria Invoice
- ☐ Orçamento aprovado aloca equipamentos
- ☐ Orçamento aprovado cria Tour
- ☐ Nenhuma funcionalidade antiga quebrou

12.3 Risco e Mitigação

Risco	Probabilidade	Impacto	Mitigação
Quebrar código existente	Média	ALTO	Criar tudo em novos arquivos, migrations seguras, testes
Cálculos errados (INSS/IRRF)	Baixa	ALTO	Validar com contador, tabelas oficiais 2025
Performance com muitos dados	Média	Médio	Índices no BD, paginação, cache
Cronjobs não executam	Baixa	Médio	Logs detalhados, monitoramento
IA recomenda errado	Média	Baixo	Marcar como “sugestão”, decisão final é do usuário

13. ANEXOS

13.1 Estrutura Final de Arquivos

```
/project
└─ main.py (expandido com migrations)
```

- models/
 - expense.py (NOVO)
 - employee.py (NOVO)
 - lead.py (NOVO)
 - maintenance.py (EXPANDIDO)
 - [outros existentes...]
- routes/
 - expense.py (NOVO)
 - rh.py (NOVO)
 - leads.py (NOVO)
 - financial.py (EXPANDIDO)
 - [outros existentes...]
- services/
 - financial_intelligence.py (NOVO)
 - commercial_intelligence.py (NOVO)
 - predictive_analytics.py (NOVO)
 - automations.py (NOVO)
 - rh_analytics.py (NOVO)
 - maintenance_analytics.py (NOVO)
 - [outros existentes...]
- jobs/
 - scheduler.py (NOVO)
- templates/
 - expense/ (NOVO)
 - rh/ (NOVO)
 - leads/ (NOVO)
 - financial/ (EXPANDIDO)
 - components/ (NOVO)
- static/
 - js/
 - modules/ (NOVO)

13.2 APIs REST (opcional para futuro app mobile)

```
GET /api/financial/fluxo-caixa?dias=15
GET /api/financial/estrategias-pagamento
GET /api/financial/dre?mes=6&ano=2025
GET /api/leads/funil
GET /api/leads/:id/estrategias
POST /api/leads/:id/reactivar
GET /api/rh/ranking?mes=6&ano=2025
GET /api/expense/contas-pagar?status=provisionado
POST /api/expense/pagar/:id
```

13.3 Glossário

Provisionamento: Criação automática de despesas futuras baseadas em histórico **Score de Intenção:** Pontuação 0-100 que indica probabilidade de lead fechar **Temperatura:** Classificação visual (frio/morno/quente) baseada no score **Folha CLT:** Pagamento mensal de funcionários com carteira assinada **Freelancer/PJ:** Profissional autônomo sem vínculo CLT **DRE:** Demonstração do Resultado do Exercício (receitas - despesas = lucro) **YoY:** Year-over-Year (comparação com mesmo período ano anterior) **MoM:** Month-over-Month (comparação mês a mês) **WoW:** Week-over-Week (comparação semana a semana)

ASSINATURAS E APROVAÇÕES

Documento preparado por: Claude (Anthropic)

Data: 25/12/2025

Versão: 1.0

Para aprovação por: Emerson - HARDCASE

